

Problem Description

Jenish and Neha are excellent friends. After performing several queries, Neha challenges Jenish to determine the highest possible Rating of the provided array A . According to Neha, the highest occurrence of an element in an array is its rating.

Jenish is given M and Q , the Magical Numbers. Jenish may perform Addition or Subtraction with M at most Q times for each element in the supplied Array A .

Because Jenish is stumped and unable to discover a solution, assist him in determining the highest possible ratings for the given array after applying queries to each element.

Constraints :

- $1 \leq N \leq 1000000$
- $1 \leq M \leq 100$
- $1 \leq Q \leq 10$
- $1 \leq A_i \leq 1000000$

Input :

- First line of Input contains integer M
- Second line contains integer Q
- Third line contains integer N
- Fourth line contains N elements representing elements of array A

Output :

- Output the highest possible rating of array A after applying Queries.

Explanation for test case 1

Jenish can add 1 in 1st element and subtract 1 from 3rd element to get highest frequency of 2 , i.e. 3

Logical Test Cases

Test Case 1

INPUT (STDIN)

1
1
4
1 2 3 4

EXPECTED OUTPUT

3

Test Case 2

INPUT (STDIN)

1
1
5
11 12 31 41 51

EXPECTED OUTPUT

2

Mandatory Test Cases

Test Case 1

KEYWORD

for(i=0;i<N;i++)

Test Case 2

KEYWORD

hash[A[i]]++;

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

9

Test Case 2

TOKEN COUNT

300

Test Case 3

NLOC

48

Code Editor

c

1 #include <stdio.h>
2 int main()

Custom Input (stdin)

T1T2

CH.SC.U4CSE25225 - M ASHWIN | Department of Computer Science & Engineering (CSE) | 2026

```
Hashing6.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp_int(const void *a, const void *b) {
5     return (*(const int *)a - *(const int *)b);
6 }
7
8 int main() {
9     int M, Q, N;
10    if (scanf("%d %d %d", &M, &Q, &N) != 3) return 0;
11
12    int *A = (int *)malloc(N * sizeof(int));
13    int *hash = (int *)calloc(1000005, sizeof(int));
14    int i;
15    for(i=0;i<N;i++) {
16        scanf("%d", &A[i]);
17        hash[A[i]]++;
18    }
19
20    int **groups = (int **)malloc(M * sizeof(int *));
21    int *group_sizes = (int *)calloc(M, sizeof(int));
22    int *group_caps = (int *)calloc(M, sizeof(int));
23
24    for (i = 0; i < M; i++) {
25        group_caps[i] = 16;
```

```
26     groups[i] = (int *)malloc(group_caps[i] * sizeof(int));
27 }
28
29 for (i = 0; i < N; i++) {
30     int rem = A[i] % M;
31     int q = A[i] / M;
32     if (group_sizes[rem] == group_caps[rem]) {
33         group_caps[rem] *= 2;
34         groups[rem] = (int *)realloc(groups[rem], group_caps[rem] * sizeof(int));
```

main | CH.SC.U4CSE25225 | Hashing6.c

Ln 68, Col 1 Spaces: 4 UTF-8 CRLF C

```
36     groups[rem][group_sizes[rem]++] = q;
37 }
38
39 int max_rating = 0;
40 for (int r = 0; r < M; r++) {
```

```
41     if (group_sizes[r] == 0) continue;
42     qsort(groups[r], group_sizes[r], sizeof(int), cmp_int);
43
44     int left = 0;
45     for (int right = 0; right < group_sizes[r]; right++) {
46         while (groups[r][right] - groups[r][left] > 2 * Q) {
47             left++;
48         }
49         int current_len = right - left + 1;
50         if (current_len > max_rating) {
51             max_rating = current_len;
52         }
53     }
54 }
```

```
55
56     printf("%d\n", max_rating);
57
58     for (i = 0; i < M; i++) {
59         free(groups[i]);
60     }
61     free(groups);
62     free(group_sizes);
63     free(group_caps);
64     free(hash);
65     free(A);
66     return 0;
67}
68
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

1

1

4

1 2 3 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing6.exe

3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

1

1

5

11 12 31 41 51

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Hashing6.exe

2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>